

Projektarbeit: Der Go Typechecker

- Der Go Typechecker
 - Einführung
 - Bausteine des Typecheckers
 - Wichtige `types.Object`-Implementierungen
 - Organisation von Objekten
 - Scope Hierarchie
 - Namensauflösung
 - Typen
 - Wichtige `types.Type S`
 - Komaptibilität von Typen
 - Zuweisbarkeit
 - Vergleichbarkeit
 - Umwandlungsfähigkeit
 - Implementierungsdetails des Go Typecheckers
 - Verbindung von AST und Typechecker
 - Verbesserung von Typeerrors
 - Generics in Go's Typensystem
 - Konsequenzen des Monomorphisierungsansatzes
 - Kontrast zu anderen Sprachen
 - Ablauf des Typecheckings
 - Ergebnis des Typechecker
- Mehrere Typfehler eines Programms finden
 - Die Idee
 - Implementierung + Probleme und Einschränkungen
- Quellen und weiterführende Literatur
- Angaben zur Nutzung von KI

Der Go Typechecker

Einführung

- Der Go Typechecker ist ein wesentlicher Bestandteil des Go-Compilers, der sicherstellt, dass der Code den Typregeln der Sprache entspricht.

- Go stellt im package `go/types` genau diese Funktionalität bereit, die es ermöglicht, Go-Code zu analysieren und zu überprüfen.
- Kernaufgaben des Typecheckers:
 - **Identifizierung Resolution:** Verknüpft Bezeichner (Variablen-, Funktions- und Typnamen) mit ihren Deklarationen. Zum Beispiel: `fmt.Println` -> welches `Println`?
 - **Type Deduction:** Bestimmt die Typen von Ausdrücken basierend auf ihren Operanden und Kontext und stellt sicher, dass die Typen kompatibel sind. Z.B.: `a + b` -> welcher Typ?
 - **Constant Evaluation:** Berechnet die Werte von Konstanten zur Compile-Zeit. > Achtung: Diese 3 Aufgaben sind stark miteinander verknüpft und müssen daher zusammen durchgeführt werden. Zum Beispiel kann der Typ eines Ausdrucks von einer Konstanten abhängen.
- Zentrale Datenstrukturen des Typecheckers:
 - **Objekte** (`types.Object`): Repräsentieren deklarierte Entitäten wie Variablen, Funktionen, Typen und Pakete.
 - **Typen** (`types.Type`): Repräsentieren die verschiedenen Typen in Go, einschließlich primitiver Typen, zusammengesetzter Typen (Structs, Slices, Maps) und generischer Typen mit Typparametern.
 - **Scopes** (`types.Scope`): Repräsentieren ein mapping von Bezeichnern zu Objekten in einem bestimmten Gültigkeitsbereich (z.B. Paket-, Funktions- oder Blockebene).

Bausteine des Typecheckers

Jedes deklarierte Element in Go wird im Typechecker durch ein `types.Object` repräsentiert. Dieses wird verwendet, um Informationen über das deklarierte Element zu speichern und darauf zuzugreifen. Beispielsweise kann dadurch im Fall von Fehler- oder Code-Analyse-Tools auf Metadaten und präzise Positionen von Deklarationen im Quellcode zugegriffen werden. Das `types.Object`-Interface setzt u.A. folgende Methoden voraus (Auswahl):

- `Name() string`: Gibt den Namen des Objekts zurück (z.B. den Variablennamen).
- `Exported() bool`: Gibt zurück, ob das Objekt exportiert ist (d.h. ob es mit einem Großbuchstaben beginnt).
- `Type() Type`: Gibt den Typ des Objekts zurück (z.B. den Typ einer Variable oder die signatur einer Funktion).
- `Pos() token.Pos`: Gibt die Position der Deklaration des Objekts im Quellcode zurück.

- `Parent() *Scope` : Gibt den Gültigkeitsbereich zurück, in dem das Objekt deklariert ist (z.B. Paket- oder Funktionsscope).
- `Pkg() *Package` : Gibt das Paket zurück, zu dem das Objekt gehört. `nil` für Objekte im `universe`-Scope (vordefinierte Typen und Funktionen).
- `Id() string` : Gibt eine eindeutige Kennung für das Objekt zurück. Zwei IDs sind genau dann verschieden, wenn diese unterschiedliche Namen habend, oder in unterschiedlichen Paketen deklariert und nicht exportiert sind (*Uniqueness of identifiers*). Für *nicht* exportierte Objekte wird daher die Paketkennung in die ID einbezogen, um Kollisionen zu vermeiden.

Wichtige `types.Object` -Implementierungen

- `*types.Var` : Repräsentiert eine Variable (lokal, global oder Feld in einem Struct).
- `*types.Func` : Repräsentiert eine Funktion oder Methode.
- `*types.TypeName` : Repräsentiert einen benutzerdefinierten Typ (z.B. `type MyInt int`, `type MyStruct struct {...}`).
- `*types.Const` : Repräsentiert eine Konstante (z.B. `const Pi = 3.14`).
- `*types.PkgName` : Repräsentiert den Import eines Pakets in einem anderen Paket (z.B. das `fmt` in `import "fmt"`).

Objekte sind kanonisch, d.h. es gibt genau ein `types.Object` für jede deklarierte Entität im Quellcode. Dies ermöglicht eine konsistente und effiziente Verwaltung von Typinformationen während der Typechecking-Phase.

Organisation von Objekten

Der Go Typechecker verwendet Scopes (`types.Scope`), um Objekte zu organisieren und den Gültigkeitsbereich von Bezeichnern zu verwalten. Jeder Scope wird durch einen lexikalischen Block im Quellcode beschrieben (z.B. ein Paket, eine Funktion oder ein Codeblock), in dem Bezeichner deklariert und verwendet werden können.

Scope Hierarchie

Scopes sind hierarchisch organisiert, wobei jeder i.d.R. jeder Scope einen übergeordneten Scope hat.

1. **`universe` -Scope (root)**: Der globale `types.Universe` -Scope enthält vordefinierte Typen und Funktionen (z.B. `int`, `string`, `println`). Dieser sollte niemals verändert werden. Der `universe` -Scope hat als einziger Scope keinen übergeordneten Scope.

2. **package -Scope**: Jeder Paket-Scope enthält alle Objekte, die in einem bestimmten Paket deklariert sind. Jedes Paket hat seinen eigenen Scope, und hat den `universe` -Scope als übergeordnet.
3. **Datei-Scope**: Jede Quellcodedatei (`*ast.File`) hat ihren eigenen Scope, der den entsprechenden Paket-Scope als übergeordneten Scope hat.
4. **Blocklevel-Scopes**: Jede Kontrollanweisung oder Funktion hat ihren eigenen Scope, der den übergeordneten Scope (z.B. Datei-Scope) als übergeordneten Scope hat. Geschachtelte Blöcke (z.B. Schleifen, `if` -Anweisungen) haben ebenfalls eigene Scopes, die den Scope der umgebenden Funktion oder des Blocks untergeordnet sind.

Namensauflösung

Um ein Objekt anhand seines Namens zu finden, stellt das `types.Scope` -Struct zwei zentrale Methoden bereit: - `Lookup(name string) *Object` : Sucht im aktuellen Scope nach einem Objekt mit dem angegebenen Namen. Wenn das Objekt nicht gefunden wird, wird `nil` zurückgegeben. - `LookupParent(name string, pos token.Pos) (*Scope, Object)` : Sucht rekursiv in dem aktuellen Scope und den übergeordneten Scopes nach einem Objekt mit dem angegebenen Namen. Der `pos` -Parameter verweist dabei auf die Position im Quellcode, an welcher nach dem Objekt gesucht werden soll. Das ist nötig, um sicherzustellen, dass ein Objekt nur gefunden wird, wenn es zum Zeitpunkt der Suche bereits deklariert wurde (Lexikalische Sichtbarkeit). Z.B. kann dadurch eine Variable im gleichen Scope nicht vor ihrer Deklaration gefunden werden.

Typen

Jedes Objekt (`types.Object`) des Go Typecheckers hat einen zugehörigen Typ (`types.Type`), der den Datentyp der deklarierten Entität beschreibt. Der `types.Type` -Interface ist die zentrale Abstraktion für alle Typen in Go, einschließlich primitiver Typen (z.B. `int` , `string`), zusammengesetzter Typen (z.B. Structs, Slices, Maps) und generischer Typen mit Typparametern. Das `types.Type` -Interface definiert nur wenige Methoden, da Typen sehr unterschiedlich sein können. Die primäre Methode ist:

- `Underlying() Type` : Gibt den zugrunde liegenden Typ zurück. Dies ist besonders nützlich für benutzerdefinierte Typen, um den Basisdatentyp zu ermitteln. Für primitive Typen gibt diese Methode den Typ selbst zurück. Zugrunde liegende Typen sind niemals benannte Typen oder Aliase.

Wichtige `types.Type` s

- `*types.Basic` : Repräsentiert primitive Typen wie `int` , `string` , `bool` .

- `*types.Struct` : Repräsentiert Struct-Typen mit Feldern.
- `*types.Interface` : Repräsentiert Interface-Typen mit Methoden.
- `*types.Signature` : Repräsentiert Funktions- und Methodensignaturen.
- `*types.Named` : Repräsentiert benannte Typen, die durch `type`-Deklarationen definiert sind.

Achtung: Nach der Go-Spezifikation sind primitive Typen wie `int` und `string` ebenfalls benannte Typen, da sie durch `type`-Deklarationen definiert sind. Im Go Typechecker werden diese jedoch als `*types.Basic` repräsentiert, um ihre spezielle Rolle als primitive Typen zu verdeutlichen.

Kompatibilität von Typen

Um zu überprüfen, ob zwei Typen miteinander kompatibel sind, unterscheidet Go zwischen drei Beziehungen von Typen. Für jede dieser Beziehungen stellt der `go/types`-Package entsprechende Funktionen bereit

Zuweisbarkeit

Zuweisbarkeit regelt, welche Paare von Typen in Zuweisungen (darunter zählen auch Funktionsaufrufe mit Parametern, Map-Zugriff, etc.) verwendet werden können. Für zwei Typen `T` und `V` ist `V` zuweisbar zu `T`, wenn eines der folgenden Kriterien erfüllt ist (Auswahl):

- `V` und `T` sind identisch.
- `V` und `T` haben den gleichen zugrunde liegenden Typ und mindestens einer von `T` oder `V` ist kein benannter Typ. > Achtung: Benannte Typen bezieht sich hier auf die Definition der Go-Spezifikation, nicht auf die vom Typechecker verwendeten `*types.Named`. Daher sind `int`, `string`, etc. auch benannte Typen.

```
type MyInt int
var a int
var b MyInt
```

`a = b` // Nicht erlaubt: auf beiden Seiten sind benannte Typen

```
type MySlice []int
var c []int
var d MySlice
```

```
c = d
// Erlaubt: zugrunde liegender Typ ist gleich ([]int) und c ist kein
// benannter Typ
```

- Weitere spezielle Regeln für bestimmte Typen (z.B. Schnittstellen, Funktionen, etc.). Um zu überprüfen, ob zwei Typen zueinander zuweisbar sind, stellt das `go/types` -Package die Funktion `types.AssignableTo(V, T Type) bool` bereit.

Vergleichbarkeit

Regelt, ob ein Type mit `==` oder `!=` verglichen werden kann. Primitive Typen und Pointer sind beispielsweise immer vergleichbar, während Structs und Arrays nur unter bestimmten Bedingungen vergleichbar sind. Damit ein Struct vergleichbar ist, müssen alle seine Felder vergleichbar sein. Arrays sind vergleichbar, wenn ihr Elementtyp vergleichbar ist. Slices, Maps und Funktionen sind niemals vergleichbar.

Um zu überprüfen, ob ein Typ vergleichbar ist, stellt das `go/types` -Package die Funktion `types.Comparable(T Type) bool` bereit.

Umwandlungsfähigkeit

Regelt, ob ein Wert von einem Type in einen anderen Type umgewandelt werden kann. Umwandlungen können sowohl explizit (z.B. `T(v)`) als auch implizit (z.B. bei Funktionsaufrufen) erfolgen. Ein Wert `x` kann dann in einen Typ `T` umgewandelt werden, wenn eines der folgenden Kriterien erfüllt ist (Auswahl):

- `x` ist zuweisbar zu `T`.
- `x` ist ein `string` und `T` ist ein `[]byte` oder `[]rune` (undumgekehrt).
- `x` und `T` sind beide numerische Typen (z.B. `int`, `float64`, etc.).
- Weitere spezielle Regeln für bestimmte Typen (z.B. Typeparameter, Pointer, etc.).

Um zu überprüfen, ob ein Typ in einen anderen Typ umgewandelt werden kann, stellt das `go/types` -Package die Funktion `types.ConvertibleTo(V, T Type) bool` bereit.

Implementierungsdetails des Go Typecheckers

Der Go Typechecker lädt zu Beginn eines Checks alle Objekte und Typen in den Arbeitsspeicher. Zuweisbarkeit, Vergleichbarkeit und Umwandlungsfähigkeit wird dabei **nicht** gecached. Beispielsweise wird bei jeder Struct-zu-Interface-Zuweisung erneut überprüft, ob das Struct alle geforderten Methoden des Interfaces implementiert. Da bereits alle Objekte im Speicher vorliegen, ist dies jedoch sehr schnell.

In einem kurzen Experiment lässt sich darstellen, dass der Go Typechecker Zuweisbarkeitsprüfungen nicht cached. Siehe dazu den Ordner `stress-test` für weitere Informationen.

Verbindung von AST und Typechecker

Mit dem `types.Type` und `types.Object` fehlt dem Typechecker noch die Verbindung zum Quellcode. Diese Verbindung wird durch das `types.Info`-Struct hergestellt, das während des Typecheckings mit Informationen über die Typen und Objekte im Quellcode gefüllt wird. Das `types.Info`-Struct enthält mehrere Maps, die verschiedene Aspekte des Quellcodes abbilden. Die wichtigsten davon sind:

- `Types map[ast.Expr]types.TypeAndValue` : Verknüpft jeden AST-Ausdruck (`ast.Expr`) mit seinem Typ und Wert (nur für Konstanten).
- `Defs map[*ast.Ident]types.Object` : Verknüpft jede Identifier-Deklaration (`*ast.Ident`) mit dem entsprechenden Objekt (`types.Object`).
- `Uses map[*ast.Ident]types.Object` : Verknüpft jede Identifier-Verwendung (`*ast.Ident`) mit dem entsprechenden Objekt (`types.Object`).
- `Scopes map[ast.Node]*types.Scope` : Verknüpft jeden AST-Knoten, der einen Gültigkeitsbereich definiert (z.B. Funktionen, Blöcke), mit dem entsprechenden Scope (`types.Scope`).

Mit diesen Feldern ist das `types.Info`-Struct die zentrale Verbindung zwischen dem abstrakten Syntaxbaum (AST) und den Typinformationen, die vom Typechecker generiert werden. Dadurch können Tools und Anwendungen detaillierte Analysen des Go-Codes durchführen, indem sie sowohl die Struktur des Codes als auch die zugehörigen Typinformationen berücksichtigen.

Verbesserung von Typeerrors

Der Go Compiler gibt bei Typfehlern nur sehr allgemeine Fehlermeldungen aus, die oft wenig hilfreich sind, um die genaue Ursache des Problems zu identifizieren. Durch die Verwendung des `go/types`-Packages könnte es möglich sein, detailliertere und präzisere Fehlermeldungen zu generieren. Eine einfache Implementierung könnte zunächst den Context der Fehlerstelle dargestellt werden. Ein kleines Beispiel befindet sich im `better-error` Ordner:

```
cd better-error
go run better_error.go error_prog.gotest
```

```
Type error in .\error_prog.gotest at line 28, column 11:
```

```
18 | func (x Sb) m1() {}
19 | func (x Sb) m2() {}
20 |
21 | // Some generic function.
22 | func f[T A](x T, y B) {
23 |     y.m2()
24 |     x.m1()
25 | }
26 |
27 | func main() {
> 28 |     f[Sb](Sa{}, Sb{})
    |           ^^---- Here
29 | }
30 |
```

```
Error: cannot use Sa{} (value of struct type Sa) as Sb value in argument to
```

Dies könnte nun erweitert werden, um über die AST-Struktur weitere Hinweise zu geben, z.B. welche Methoden fehlen, welche Typen erwartet wurden, etc.

Generics in Go's Typensystem

Mit der Einführung von Generics in Go 1.18 wurde das Typensystem von Go erheblich erweitert. Generics ermöglichen es, Funktionen und Datentypen zu definieren, die mit verschiedenen Typen arbeiten können, ohne dass der Code für jeden Typ dupliziert werden muss. Dies wird durch die Verwendung von Typparametern erreicht, die als Platzhalter für konkrete Typen dienen. Für eine genauere Beschreibung von Generis, siehe [generics.md](#).

Für die Implementierung von Generics verwendet der Go Compiler Monomorphisierung. Das bedeutet, dass für jede Instanziierung einer generischen Funktion oder eines generischen Typs eine sperate Version des Codes mit dem konkreten Typ generiert wird. Generics werden also zur Compile-Zeit aufgelöst und fungieren damit lediglich als eine Art Kurzschreibweise für wiederverwendbaren Code.

Konsequenzen des Monomorphisierungsansatzes

- **Kein Laufzeit-Overhead:** Da Generics zur Compile-Zeit aufgelöst werden, gibt es keinen Laufzeit-Overhead durch Typparameter oder Typinformationen. Der generische Code wird in spezialisierten Versionen für jeden konkreten Typ kompiliert, was zu optimiertem Maschinencode führt.

- **Code-Duplizierung / Binärgrößenzunahme:** Jede Instanziierung einer generischen Funktion oder eines generischen Typs führt zu einer separaten Kopie des Codes. Dies kann zu einer erhöhten Binärgröße führen, insbesondere wenn viele verschiedene Typen verwendet werden.
- **Eingeschränkte Zuweisbarkeit trotz Constraint:** Da Generics zur Compile-Zeit aufgelöst werden, verschwinden Constraints nach der Instanziierung. Dadurch sind z.B. Zuweisungen eines konkreten Structs zu einer Generischen Variable, wessen Typ durch ein Interface-Constraint eingeschränkt ist, nicht erlaubt, selbst wenn das Struct alle Methoden des Interface implementiert:

```
type Shape interface {
    size()
}

type Square struct{}
func (m Square) size() {}

type Circle struct{}
func (m Circle) size() {}

func f[T Shape](x T, y Circle) {
    x = y // Nicht erlaubt, da T nach der Instanziierung kein
          Interface mehr ist
}

...

func main() {
    f[Square](Square{}, Circle{}) // Aufruf der generischen Funktion
}
```

Der Go-Compiler überstzt die generische Funktion dann in etwa so:

```
func f_Square(x Square, y Circle) {
    x = y // Nicht erlaubt, da Square nicht Circle und auch kein
          subtype davon ist
}
```

Wie im Beispiel zu sehen, verschwindet das Interface-Constraint `Shape` nach der Instanziierung, und `T` wird zu `Square`. Daher ist die Zuweisung von `y` (vom Typ `Circle`) zu `x` (vom Typ `Square`) nicht erlaubt, da `Square` und `Circle` unterschiedliche Typen sind und keine direkte Zuweisbarkeit besteht.

Kontrast zu anderen Sprachen

Viele Programmiersprachen, welche Generics unterstützen, verwenden stattdessen Typ-Erasure. Dabei werden Typparameter zur Laufzeit durch einen allgemeinen Typ (z.B. `Object` in Java) ersetzt, und Typinformationen gehen verloren. Dies ermöglicht eine flexiblere Zuweisbarkeit und Interoperabilität zwischen verschiedenen Typen, führt jedoch zu Laufzeit-Overhead und potenziellen Performance-Einbußen.

Der Vorteil von Go's Monomorphisierungsansatz liegt in der Kombination aus hoher Performance (kein Laufzeit-Overhead) und Typsicherheit zur Compile-Zeit. Allerdings erfordert dieser Ansatz ein tieferes Verständnis der Typensystemregeln (was zu Verwirrungen führen kann) und kann zu größeren Binärdateien führen. **Der Nachteil** ist die eingeschränkte Flexibilität bei der Zuweisbarkeit und die Notwendigkeit, den Code für verschiedene Typen zu duplizieren.

Ablauf des Typecheckings

Der Go Typechecker überprüft eine oder mehrere Go-Dateien, repräsentiert als ASTs (`*ast.File`). Dabei durchläuft der Typechecker mehrere Phasen, um sicherzustellen, dass der Code den Typregeln von Go entspricht. Ein grober Überblick über den Ablauf des Typecheckings ist wie folgt (parallel zur tatsächlichen Implementierung in `go/types/check.go`):

1. **Initialize Files:** Initialisiert die internen Datenstrukturen des Typecheckers für die zu überprüfenden Dateien. Dies umfasst u.A. das Erfassen von Go-Versionen pro Datei, um sicherzustellen, dass alle Dateien miteinander kompatibel sind.
2. **Collect Objects:** Alle Typechecker-Objekte (`types.Object`) werden gesammelt und in den entsprechenden Scopes (`types.Scope`) organisiert. Dies umfasst das Verarbeiten von Paket-Imports, Deklarationen von Variablen, Funktionen, Typen und Konstanten.
3. **Package Objects:** Nachdem alle Objekte gesammelt wurden, werden die ersten Paket-Objekte gechecked. Dies umfasst alle Paket Deklarationen, außer Funktionen und Methoden. Die tatsächliche Typechecking-Logik wird jedoch schon vorbereitet, um im nächsten Schritt angewendet zu werden.
4. **Process Delayed:** Die in Schritt 3 verzögerte Typechecking-Logik wird nun angewendet. Dies umfasst das Typechecking von Funktionen, Methoden und anderen Konstrukten, die auf bereits deklarierten Objekten basieren.

Ergebnis des Typechecker

Der Typechecker liefert, falls zutreffend, einen Fehler vom Typ `types.Error` zurück. Leider sind die Fehlermeldungen des Go Typecheckers oft sehr allgemein gehalten und bieten wenig Kontext zur eigentlichen Ursache des Problems. Eine genaue Unterscheidung zwischen verschiedenen Arten von Typefehlern (z.B. unbekannte Typen, nicht deklarierte Variablen, nicht-lineare Parameter) ist anhand der Fehlermeldung allein oft schwierig.

Typische Beispiele für Typefehler sind:

```
func foo(t UnknownType) {  
    print(t)  
}  
  
// Fehler: undefined: UnknownType
```

```
func foo() int {  
    return x  
}  
  
// Fehler: undefined: x
```

```
func foo(x int, x string) {  
    print(x)  
}  
  
// Fehler: x redeclared in this block
```

Mehrere Typefehler eines Programms finden

Eine Schwierigkeit bei der Verwendung des Go Typecheckers besteht darin, dass dieser maximal einen Fehler pro Durchlauf meldet. Dies liegt daran, dass der Typechecker bei einem Fehler den aktuellen Überprüfungsprozess abbricht, um inkonsistente Zustände zu vermeiden. Wenn man also mehrere Typefehler in einem Programm finden möchte, muss man sich etwas einfallen lassen.

Die Idee

Um mehrere Typefehler in einem einzigen Programm zu finden, lässt man zunächst den Typechecker normal laufen. Wenn ein Fehler gefunden wird, wird die Funktion ermittelt, in

der der Fehler aufgetreten ist. Anschließend wird eine neue Version des Programmes erstellt, in welcher die fehlerhafte Funktion durch eine Dummy-Implementierung ersetzt wird. Anschließend wird der Typechecker erneut auf das modifizierte Programm angewendet. Dieser Prozess wird wiederholt, bis keine weiteren Fehler mehr gefunden werden.

Implementierung

Eine einfache Implementierung dieser Idee befindet sich im Ordner `typecheckall`.

```
cd typecheckall
go run main.go errorprog.gotest
```

```
Type error in function `m1` at line 18, column 7: cannot use 567 (untyped i
    s = 567
        ^
Type error in function `e1` at line 37, column 7: cannot use 123 (untyped i
    s = 123
        ^
Type error in function `e3` at line 41, column 3: undefined: asd
    asd = 123
    ^
Type error in function `e2` at line 48, column 7: cannot use 312 (untyped i
    t = 312
        ^
Type error in function `e4` at line 54, column 9: cannot use Sa{} (value of
    f[Sb](Sa{}, Sb{}))
            ^
```

Im `checker`-Unterpaket befinden sich die eigentliche Logik, welche die oben beschriebenen Schritte durchführt und dabei den AST kontinuierlich modifiziert.

1. **Initiales Typechecking:** Der Typechecker wird auf das ursprüngliche Programm angewendet.
2. **Fehlererkennung:** Wenn ein Typfehler gefunden wird, wird die Funktion ermittelt, in der der Fehler aufgetreten ist. Die Position innerhalb der Funktion, sowie der Funktionsname werden gespeichert.
3. **Funktion ersetzen:** Die fehlerhafte Funktion wird durch eine Dummy-Implementierung ersetzt. Diese Dummy-Implementierung hat die gleiche Signatur wie die Originalfunktion, enthält jedoch keinen Code, außer einem `return`-Statement (falls erforderlich). Um passende Werte für das Return-Statement zu generieren, wird Gos `new`-Funktion verwendet, welche für einen gegebenen Typ

einen Nullwert erzeugt. Dies funktioniert sogar für Interfaces, was die Implementierung vereinfacht.

4. **Erneutes Typechecking:** Der Typechecker wird erneut auf das modifizierte Programm angewendet.
5. **Wiederholung:** Die Schritte 2-4 werden wiederholt, bis keine weiteren Typefehler mehr gefunden werden.
6. **Fehlerausgabe:** Alle gefundenen Typefehler werden gesammelt und am Ende ausgegeben. Durch den gespeicherten Funktionsnamen und die Position kann die Fehlerstelle im Originalcode leicht gefunden werden.

Probleme und Einschränkungen

- **Entstehende Fehler durch das Ersetzen von Funktionen:** Wenn eine Funktion ersetzt wird, kann dies zu neuen Typefehlern führen, die zuvor nicht aufgetreten sind. Z.B. wenn eine fehlerhafte Funktion der einzige Ort war, an dem ein importiertes Paket verwendet wurde, und die Dummy-Implementierung dieses Paket nicht mehr verwendet. In diesem Fall würde der Typechecker einen Fehler melden, dass das Paket importiert, aber nicht verwendet wird. Dies kann jedoch durch einen Konfigurationsparameter des Typecheckers `DisableUnusedImportCheck` verhindert werden.
- **Fehler in Funktionssignaturen:** Wenn der Typefehler in der Signatur einer Funktion auftritt, kann die Funktion nicht korrekt ersetzt werden. Um eine Endlosschleife zu vermeiden, wird intern eine Liste von bereits ersetzten Funktionen geführt. Bei einem erneuten Fehler in der gleichen Funktion das Programm abgebrochen.
- **Fehler außerhalb von Funktionen:** Typefehler, die außerhalb von Funktionen auftreten (z.B. in globalen Variablen oder Konstanten), können nicht durch das Ersetzen von Funktionen behoben werden. Das Programm wird in diesem Fall ebenfalls abgebrochen.
- **Mehrere Fehler in einer Funktion:** Wenn mehrere Typefehler in der gleichen Funktion auftreten, wird nur der erste Fehler gefunden. Dies ist eine bewusste Limitierung, um die Komplexität der Implementierung zu reduzieren. Theoretisch wäre aber eine Variante denkbar, welche nur Teile einer Funktion ersetzt, um mehrere Fehler in der gleichen Funktion zu finden.
- **Unterscheidung von Fehlern:** Der Typechecker meldet nur sehr generische Fehlermeldungen, ohne genaue Ursache. Z.B. können unbekannte Typen (`func foo(t UnknownType)`), nicht deklarierte Variablen (`func foo() int { return x }`), und nicht-lineare Parameter (`func foo(x int, x string)`) nur anhand der Fehlermeldung schwer unterschieden werden.

Quellen und weiterführende Literatur

- [Tutorial: Getting started with generics](#)
- [go/types](#) : The Go Type Checker
- [Updating tools to support type parameters](#)
- [go/types](#) package
- [The Go Programming Language Specification](#)
- [Go Standard Library Documentation](#) ([go/types](#))

Angaben zur Nutzung von KI

Einige Inhalte dieses Dokuments wurden mit Unterstützung von KI-Technologien recherchiert und verfasst. Dabei kamen insbesondere Sprachmodelle wie Google Gemini und Copilot zum Einsatz. Diese Technologien halfen dabei, Informationen zu strukturieren, Codebeispiele zu generieren und komplexe Konzepte verständlich darzustellen. Trotz sorgfältiger Überprüfung durch den Autor können Fehler oder Ungenauigkeiten nicht vollständig ausgeschlossen werden. Der Autor übernimmt die volle Verantwortung für den Inhalt dieses Dokuments.